

Esercizio 1

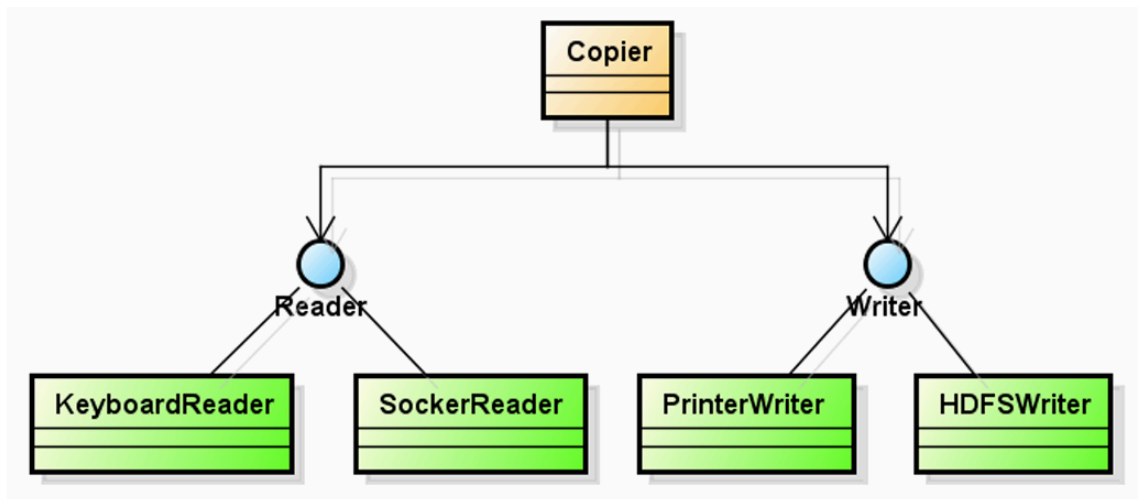
Descrizione Siano prese in considerazione le seguenti classi, che permettono di copiare dei semplici caratteri inseriti da tastiera direttamente verso una stampante.

```
public class Copier {  
    private KeyboardReader reader;  
    private PrintWriter writer;  
    public Copier(KeyboardReader reader, PrintWriter writer) {  
        this.reader = reader;  
        this.writer = writer;  
    }  
    public void copy(OutputDevice dev) {  
        char c;  
        while ((c = this.reader.read()) != EOF)  
            if (dev == PRINTER)  
                writer.write(c);  
    }  
}  
  
public class KeyboardReader {  
    public char read() {  
        // Reads a character from the keyboard  
    }  
}  
  
public class PrintWriter {  
    public void write(char c) {  
        // Writes the character directly into the printer spool  
    }  
}
```

Si fornisca il diagramma delle classi di una soluzione che permetta di estendere le suddette classi in modo tale che la classe Copy possa essere riutilizzata per copiare caratteri da provenienti da un Socket direttamente su un file all'interno di un filesystem HDFS. La soluzione dovrà tenere in considerazione il Dependency Inversion Principle.

Soluzione

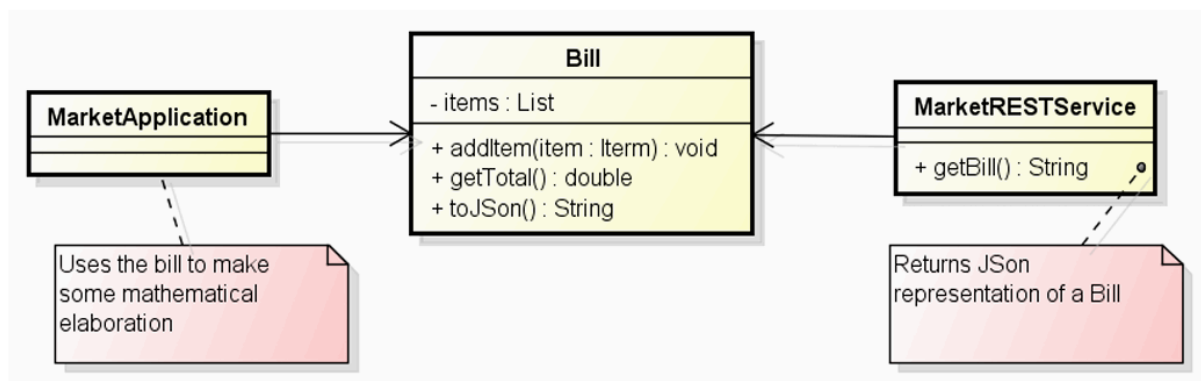
I livelli più alti dell'architettura non devono dipendere dalle implementazioni dei livelli più bassi. Copier ora dipende solo da astrazioni, così come i reader e writer concreti.



Esercizio 2

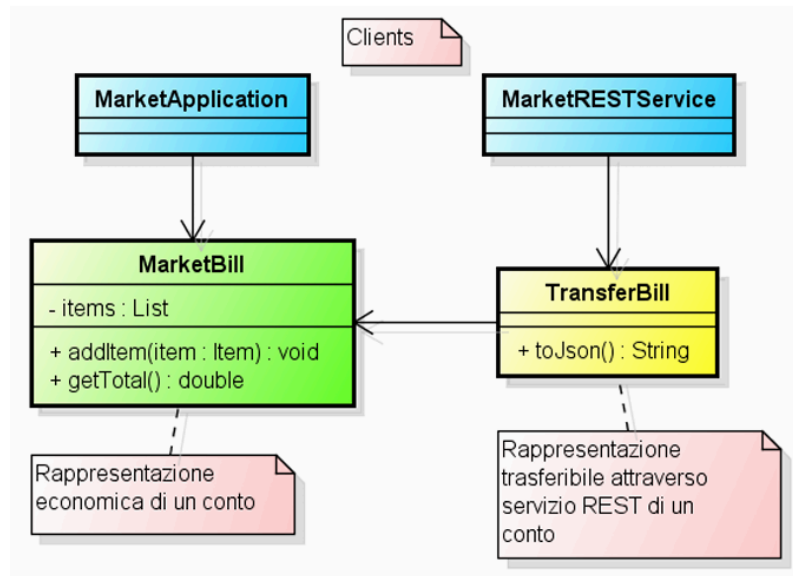
Descrizione

La classe `Bill` è utilizzata per gestire oggetti che rappresentano liste della spesa sia dal punto di vista del calcolo matematico, sia da una applicazione che espone queste informazioni in formato `Json` tramite un servizio `REST`. Dato il seguente diagramma delle classi, se ne fornisca una versione modificata in modo tale che la soluzione aderisca al "Single Responsibility Principle".



Soluzione

Le funzionalità del tipo `Bill` devono essere suddivise in opportuni tipi, in modo da suddividere gli assi di cambiamento. In questo modo, se la rappresentazione `Json` di un conto dovesse cambiare, `MarketBill` non verrebbe modificato. D'altra parte, se l'algoritmo di calcolo del totale dovesse cambiare, `TransferBill` non verrebbe modificato.



Esercizio 3

Dato il seguente codice sorgente, derivato da una cattiva progettazione:

```
public class MacBook {
    // Attributi
```

Ingegneria del Software A.A. 2017/2018 – Appello 2018-05-14 | Esercizio 3 (3 punti)

4

```

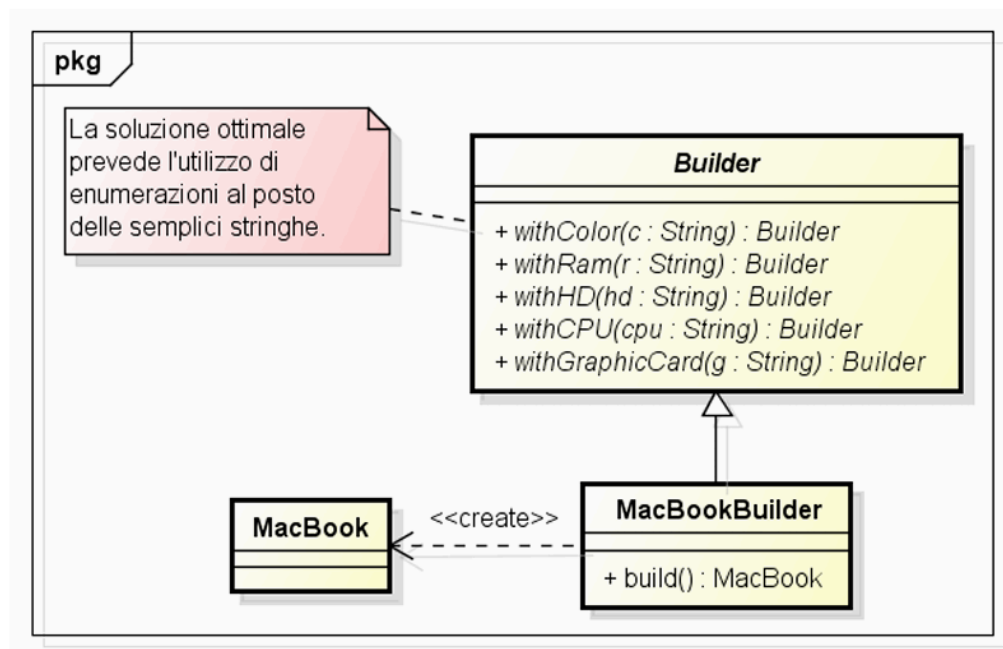
    public MacBook(String color, String ram, String harddisk,
                   String cpu, String graphicCard) { /*...*/ }
    public MacBook(String color, String ram, String harddisk,
                   String cpu) { /*...*/ }
    public MacBook(String color, String ram, String harddisk) { /*...*/ }
    public MacBook(String color) { /*...*/ }
    // Metodi
}
```

si fornisca il diagramma delle classi di una sua possibile re-ingegnerizzazione che elimini l'effetto *telescoping* attualmente presente nei costruttori, mantenendo però la caratteristica di selezione delle singole componenti della classe MacBook.

Soluzione

Soluzione

È necessario utilizzare il design patter Builder.



Esercizio 4

Sia dato il seguente frammento di codice sorgente in Java.

```
public class Animal {  
    private String animalType;  
    public static void letsTalk(Animal[] animals) {  
        for (Animal a: animals) {  
            switch (a.getAnimalType) {  
                case "Dog":  
                    ((Dog) a).bark();  
                    break;  
                case "Cat":  
                    ((Cat) a).mew();  
                    break;  
            }  
        }  
    }  
    // Other methods...  
}  
  
public class Dog {  
    // Code that properly initialize super animalType
```

```
        public void bark() { System.out.println("Bark bark"); }
    }

    public class Cat {

        // Code that properly initialize super animalType

        public void miaow() { System.out.println("Mew mew"); }
    }
```

Si modifichi il suddetto codice al fine di fare aderire il metodo `letsTalk` al principio Open-Closed.

Soluzione

È necessario uniformare i metodi nelle classi concrete in una unica interfaccia `talk`, dichiarata in `Animal`. In questo modo, è possibile estendere il sistema aggiungendo nuovi animali, senza tuttavia modificare il codice esistente.

```
public abstract class Animal {

    public abstract void talk();

    public static void letsTalk(Animal[] animals) {

        for (Animal a: animals) {

            a.talk();

        }

    }

}

public class Dog extends Animal {

    @Override

    public void talk() { System.out.println("Bark bark"); }

}

public class Cat extends Animal {

    @Override

    public void talk() { System.out.println("Mew mew"); }

}
```

Esercizio 5

Descrizione

Il *framework* degli I/O *stream* in Java ha le classi astratte `InputStream` e `OutputStream` come classi base. La componente di *input*, `InputStream`, espone un unico metodo astratto, `read() : int`, che ritorna il prossimo *byte* di dati da leggere. Usando le classi messe a disposizione da questo *framework* è possibile leggere un oggetto precedentemente salvato in un *file* compresso, utilizzando il seguente frammento di codice.

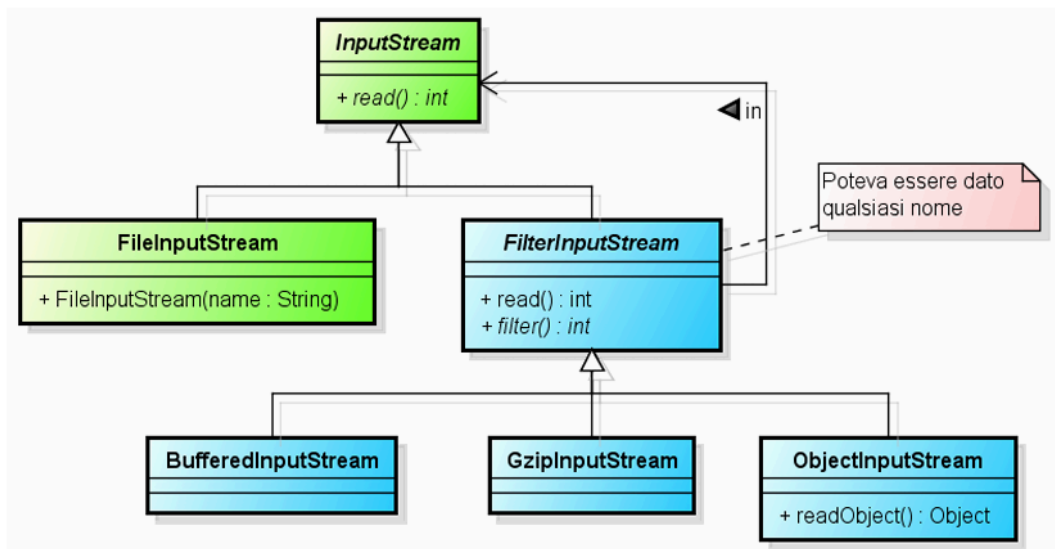
```
FileInputStream fis = new FileInputStream("/objects.gz");  
BufferedInputStream bis = new BufferedInputStream(fis);  
GzipInputStream gis = new GzipInputStream(bis);  
ObjectInputStream ois = new ObjectInputStream(gis);  
SomeObject someObject = (SomeObject) ois.readObject();
```

Il *framework* degli I/O *stream* utilizza infatti un noto *design pattern* della GoF. Si fornisca il diagramma delle classi che contestualizza il *pattern* nella gerarchia di classi sopra riportata.

Soluzione

SOLUZIONE

Il *design pattern* da utilizzare è il Decorator pattern. La classe decorator è identificata in `FilterInputStream`, ma poteva essere fornito qualsiasi nome.



Esercizio 6

L'installazione di una lista di nuovi Binding all'interno del sistema opanHAB avviene utilizzando il seguente codice Java.

```

public class Binding {
    private final String bindingType;
    public static void install(Binding[] bindings) {
        for (Binding b: bindings) {
            switch (b.bindingType) {
                case "BTicino":
                    ((BTicinoBinding) b).installBTicino();
                    break;
                case "Hue":
                    ((PhilipsHueBinding b)).installHue();
                    break;
            }
        }
    }
}

public class BTicinoBinding {
    // Code that initializes super bindingType
    public void installBTicino() {

```

```

        System.out.println("Installing BTicino binding");
    }
}

public class PhilipsHueBinding {
    // Code that initializes super bindingType
    public void installHue() {
        System.out.println("Installing Philips Hue binding");
    }
}

```

Si modifichi il suddetto codice al fine di fare aderire il metodo install al principio *Open-Closed*.

Soluzione

È necessario uniformare i metodi nelle classi concrete in una unica interfaccia install, dichiarata in Binding. In questo modo, è possibile estendere il sistema aggiungendo nuove tipologie di Binding, senza tuttavia modificare il codice esistente.

```

public abstract class Binding {
    public abstract void install();
    public static void install(Binding[] bindings) {
        for (Binding b: bindings) {
            b.install();
        }
    }
}

public class BTicinoBinding extends Binding {
    @Override
    public void install() {
        System.out.println("Installing BTicino binding");
    }
}

public class PhilipsHueBinding extends Binding {
    @Override
    public void install() {
        System.out.println("Installing Philips Hue binding");
    }
}

```

Esercizio 7

Descrizione

Il *framework* delle *Collection* in Java permette di ordinare una lista in modo arbitrario utilizzando il metodo della classe `Collections`, `static <T> void sort(List<T> list, Comparator<? super T> c)`.

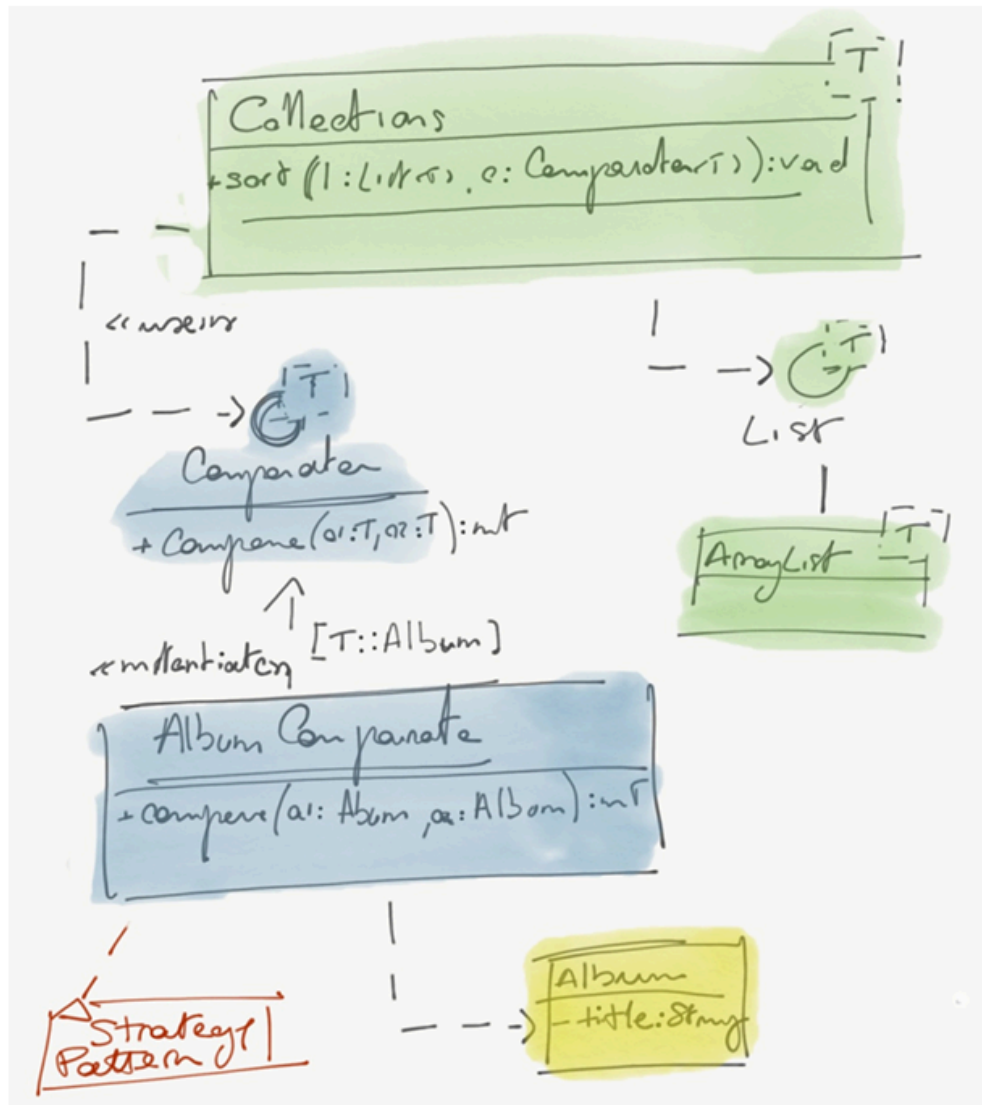
L'interfaccia `Comparator` contiene un unico metodo `int compare(T o1, T o2)`.

Tale *framework* utilizza un noto *design pattern* della GoF. Si fornisca il diagramma delle classi che lo contestualizza, per una lista di oggetti della classe `Album`, utilizzando un comparatore che ordina per `title`.

Soluzione

SOLUZIONE

Il pattern richiesto è lo Strategy pattern e il diagramma delle classi relativo all'esercizio è il seguente.

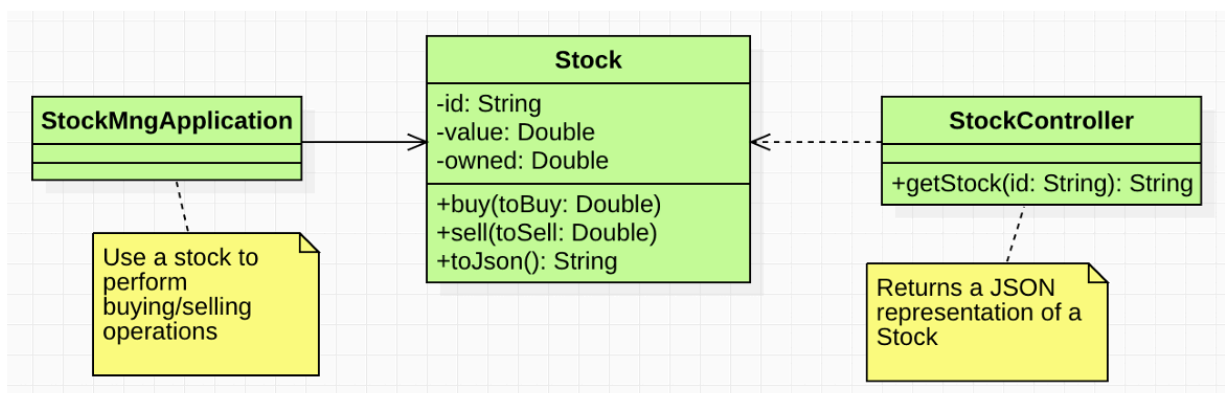


Esercizio 8

Descrizione

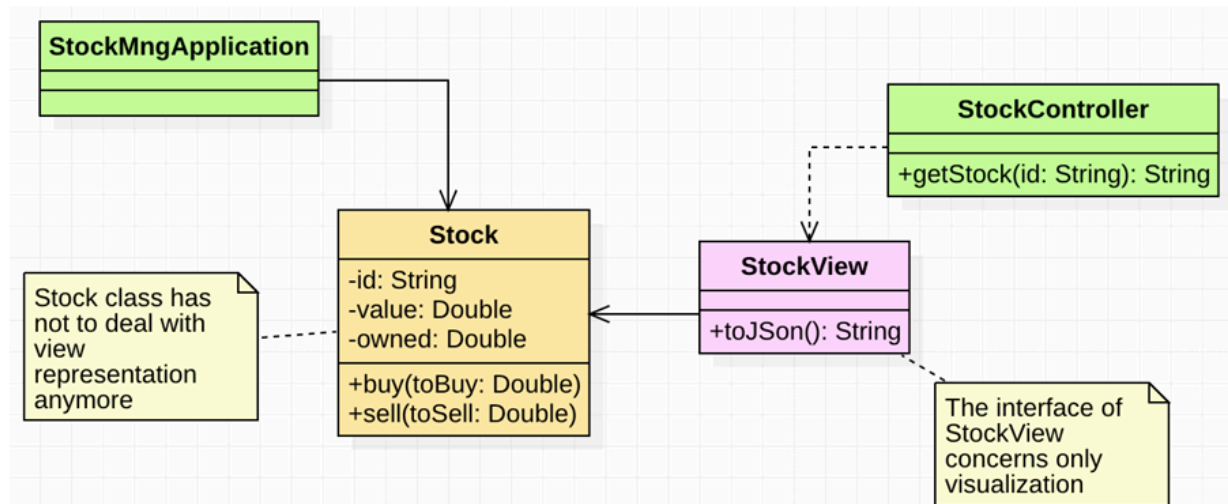
La classe `Stock` è utilizzata per gestire oggetti che rappresentano azioni finanziarie, sia come parte del processo di compravendita, che per esporre tali informazioni in formato JSON tramite un servizio REST.

Si modifichi il diagramma delle classi riportato in figura in modo da farlo aderire al "Single Responsibility Principle".



Soluzione

L'applicazione del "Single Responsibility Principle" permette di isolare le dipendenze rispetto ai client della classe `Stock`, che deve essere pertanto suddivisa in due tipi distinti.



Esercizio 9

Descrizione

Il *framework* delle *Collection* in Java permette di ordinare una lista in modo arbitrario utilizzando il metodo della classe `Collections`, `static void sort(List list, Comparator c)`. L'interfaccia `Comparator` contiene un unico metodo `int compare(T o1, T o2)`. Tale *framework* utilizza un noto *design pattern* della GoF.

Si fornisca il diagramma delle classi che lo contestualizza, per una lista di oggetti della classe `Person`, utilizzando un comparatore che ordina per `surname`.

Soluzione

Il *design pattern* utilizzato è lo *Strategy Pattern*. Di seguito si riporta il diagramma delle classi richiesto.

